Processo KDD – consulta rápida

1 · Seleção, limpeza e transformação

00 AS SEIS FASES

N3_03 · §0

Fase	Pergunta
0. Objetivo	Que decisão vai mudar? Qual é o critério de sucesso? (antes de ver os dados)
1. Seleção	Que fontes, que período, uma linha por quê?
2. Limpeza	Os dados estão corretos e completos?
3. Transformação	Estão na forma que o algoritmo precisa?
4. Redução	Precisamos de todos os atributos?
5. Mineração	Que padrões existem?
6. Avaliação	São válidos, novos, úteis e compreensíveis?

Limpeza + transformação + redução = **pré-processamento** (modelo de 5 fases). No CRISP-DM: compreensão do negócio → dos dados → preparação → modelação → avaliação → implementação. O processo é iterativo.

01 SELEÇÃO

N3_03 · §1

`pd.read_csv("clientes.csv")`	CSV exportado de um sistema.

`with sqlite3.connect("loja.db") as con:`
`  v = pd.read_sql_query(`
`  "SELECT * FROM vendas WHERE "`
`  "data BETWEEN :ini AND :fim",`
`  con, params=janela)`
	Base de dados com consulta parametrizada: os valores vão em `params`, nunca colados no SQL.

`pd.json_normalize(dados["resultados"])`	JSON aninhado (API) → tabela.
`observação: jan.–set. → atributos` `previsão: out.–dez. → alvo`	Janelas separadas: nenhum atributo pode vir de depois do alvo.

02 LIMPEZA

N3_03 · §2

`kdd.diagnosticar(df)`	Tipo, em falta, distintos e exemplos por coluna; nº de duplicados.
`kdd.remover_duplicados(df, chave="id")`	Linhas repetidas · chaves repetidas.
`kdd.uniformizar_categorias(s,` `  {"basico": "Básico"})`	Normalizar e mapear sinónimos.
`kdd.invalidar_fora_do_intervalo(df,` `  {"idade": (16, 100)})`	Valores impossíveis → NaN (regra de domínio).
`kdd.outliers_iqr(s)`	Candidatos a extremos: decidir com o negócio, não apagar às cegas.

Valores em falta: porque faltam?

Falta estrutural	Sem compras, sem contactos → o valor verdadeiro é 0: `fillna(0)`.
Valor desconhecido	Imputar **no pipeline** (mediana, mais frequente) com `add_indicator=True`.

03 TRANSFORMAÇÃO

N3_03 · §3

`df.groupby("id").agg(`
`  n=("valor", "size"),`
`  total=("valor", "sum"))`
	Agregar: uma linha por unidade (RFM: recência, frequência, valor).

`a.merge(b, on="id", how="left",` `  validate="one_to_one")`	Juntar fontes e confirmar que não há duplicação.
`train_test_split(X, y, test_size=0.25,` `  stratify=y)`	Separar o teste **antes** de aprender qualquer transformação.
`StandardScaler()` `MinMaxScaler()` `np.log1p(x)`	Padronizar (média 0, dp 1) · intervalo 0–1 · reduzir assimetria.
`OneHotEncoder(handle_unknown="ignore")` `OrdinalEncoder(categories=[ordem])`	Categorias sem ordem · com ordem.
`prep = kdd.construir_preprocessamento(` `  numericas, categoricas, ordinais)`	`ColumnTransformer`: imputar + escalar + codificar.

Processo KDD – consulta rápida

2 · Redução, mineração e avaliação

04 REDUÇÃO

N3_03 · §4

`kdd.colunas_redundantes(X, limite=0.95)`	Constantes e |correlação| > limite.
`mutual_info_classif(X_t, y)`	Informação mútua com o alvo (também relações não lineares).
`SelectKBest(mutual_info_classif, k=8)`	Seleção dentro do pipeline: mantém o significado dos atributos.
`PCA(n_components=0.90)`	Componentes que explicam 90 % da variância: comprime; exige padronização.
`pca.explained_variance_ratio_.cumsum()`	Variância explicada acumulada.

05 MINERAÇÃO

N3_03 · §5

`Pipeline([("prep", prep), ("modelo", modelo)])`	Pré-processamento e modelo juntos.
`DummyClassifier( strategy="most_frequent")`	Modelo de referência: o mínimo a bater.
`kdd.comparar_modelos(modelos, prep, X_treino, y_treino)`	Validação cruzada estratificada (`roc_auc`, `f1`, `accuracy`).
`LogisticRegression` `RandomForestClassifier` `HistGradientBoostingClassifier`	Em empate, escolher o mais simples.
`KMeans(n_clusters=4, n_init=10)` `silhouette_score(X, rotulos)`	Agrupamento: escolher k pela silhueta e pela utilidade.
`DummyRegressor  KFold` `scoring="neg_mean_absolute_error"`	Regressão: referência, partições e métricas.

! FUGAS DE INFORMAÇÃO E ARMADILHAS

Atributos do futuro	Medidos depois do alvo (ou por causa dele): resultados bons demais.
Ajustar com todos os dados	Escalar, imputar ou selecionar antes de separar o teste.
Exatidão enganadora	Com 74 % numa classe, dizer sempre essa dá 74 %. Usar AUC, F1, *lift*.
Apagar extremos às cegas	A regra IQR marca compras grandes e devoluções verdadeiras.
Espreitar o teste	Usado para escolher modelos, deixa de medir a generalização.
Ler um coeficiente sozinho	Pode ter o sinal contrário da tabela simples (paradoxo de Simpson).
Associação ≠ causa	Testar a ação com um piloto e um grupo de controlo.

06 AVALIAÇÃO

N3_03 · §6

`modelo.fit(X_treino, y_treino)` `p = modelo.predict_proba(X_teste)[:, 1]`	Treinar com todo o treino; avaliar no teste **uma vez**.
`classification_report(y_teste, prev)` `roc_auc_score(y_teste, p)`	Precisão, sensibilidade, F1 · AUC.
`kdd.lift_por_decil(y_teste, p)`	Taxa real nos 10 % de maior risco ÷ taxa média.
`permutation_importance( modelo, X_teste, y_teste)`	Quanto piora o modelo sem cada atributo.
`modelo.named_steps["modelo"].coef_`	Sinal e peso, «com os outros constantes».
Válido · novo · útil · compreensível	E acionável: leva a uma ação cujo efeito se pode medir.